

1. INTRODUCTION

A Smart Grid is an advanced electricity distribution system that uses modern technology to monitor and manage electricity efficiently. Traditional power systems face issues like power loss, overload, and inefficient distribution.

The Smart Grid Electricity Distribution System uses Graph data structures to represent the electrical network. In this graph:

- Vertices represent power stations or substations.
- Edges represent electrical connections between them.

This project helps in understanding how electricity travels from one station to another and how graph algorithms improve power management.

2. OBJECTIVES

The main objectives of the project are:

- To implement graph data structures using C language.
 - To simulate electricity distribution networks.
 - To perform BFS and DFS traversals.
 - To display connections between stations.
 - To improve understanding of real-world graph applications.
-

3. SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB minimum
- Hard Disk: 100 GB
- Keyboard and Monitor

Software Requirements

- Operating System: Windows/Linux
 - Programming Language: C
 - Compiler: GCC or Turbo C
 - IDE: VS Code / CodeBlocks
-

4. METHODOLOGY

The project uses Graph data structures implemented through an adjacency matrix.

Steps Involved

1. Create graph nodes representing stations.
2. Add edges between connected stations.
3. Display the electricity network.
4. Perform BFS traversal.
5. Perform DFS traversal.

The graph helps in visualizing electricity flow between stations.

5. ALGORITHM

BFS Algorithm

1. Start from source node.
2. Mark node as visited.
3. Insert node into queue.
4. Visit adjacent nodes.
5. Repeat until queue becomes empty.

DFS Algorithm

1. Start from source node.
2. Mark node as visited.
3. Visit adjacent unvisited nodes recursively.
4. Continue until all nodes are visited.

```
        if(graph[current][i] == 1 &&
!visited[i]) {
            visited[i] = 1;
            queue[rear++] = i;
        }
    }
}
```

```
void DFS(int node) {
    visited[node] = 1;
    printf("%d ", node);
```

```
    for(int i = 0; i < n; i++) {
        if(graph[node][i] == 1 && !visited[i]) {
            DFS(i);
        }
    }
}
```

```
int main() {
    int edges, u, v;
```

```
    printf("Enter number of stations: ");
    scanf("%d", &n);
```

```
    printf("Enter number of connections: ");
    scanf("%d", &edges);
```

```
    for(int i = 0; i < edges; i++) {
        printf("Enter connection (u v): ");
        scanf("%d %d", &u, &v);
```

```
        graph[u][v] = 1;
        graph[v][u] = 1;
    }
```

```
    printf("\nGraph Representation:\n");
```

```
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            printf("%d ", graph[i][j]);
        }
        printf("\n");
    }
```

```
    for(int i = 0; i < n; i++)
        visited[i] = 0;
```

6. SOURCE CODE

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int graph[MAX][MAX];
int visited[MAX];
int n;
```

```
void BFS(int start) {
    int queue[MAX], front = 0, rear = 0;
```

```
    visited[start] = 1;
    queue[rear++] = start;
```

```
    printf("BFS Traversal: ");
```

```
    while(front < rear) {
        int current = queue[front++];
        printf("%d ", current);
```

```
        for(int i = 0; i < n; i++) {
```

```
BFS(0);

for(int i = 0; i < n; i++)
    visited[i] = 0;

printf("\nDFS Traversal: ");
DFS(0);

return 0;
}
```

7. OUTPUT

Sample Input

Enter number of stations: 5
Enter number of connections: 4

0 1
0 2
1 3
2 4

Sample Output

BFS Traversal: 0 1 2 3 4

DFS Traversal: 0 1 3 2 4

8. ADVANTAGES

- Efficient electricity monitoring.
 - Reduces power wastage.
 - Easy network analysis.
 - Faster fault detection.
 - Real-world application of graphs.
-

9. APPLICATIONS

- Smart Cities
 - Power Distribution Systems
 - Industrial Electricity Networks
 - Energy Management Systems
-

10. FUTURE ENHANCEMENT

- Add shortest path algorithms.
 - Implement real-time monitoring.
 - Add graphical user interface.
 - Integrate IoT sensors.
-

11. CONCLUSION

The Smart Grid Electricity Distribution System demonstrates the implementation of Graph data structures using C programming. The project successfully performs electricity network representation and traversal operations using BFS and DFS algorithms. It improves understanding of smart electricity management systems and real-world graph applications.

12. REFERENCES

1. E. Balagurusamy, Programming in ANSI C.
2. Data Structures Using C – Reema Thareja.
3. IEEE Papers on Smart Grid Systems.
4. www.geeksforgeeks.org
5. C Programming Documentation.

